

---

# 第 5 章

## Matplotlib 及其基本使用

---

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

## 5. Matplotlib 及其基本使用（新版讲义）

本页是第 5 章的[章首页（索引）](#)。正文位于 01-基本绘图.md ~ 05-常见误区与技巧.md，与原版资料完全分离（原版见 ../../原始资料/chap5/）。

### 本章概览

Matplotlib 是 Python 科学计算中最常用的可视化库：它把“数据 → 图形”变成一系列可组合的对象（Figure、Axes、Label、Line、Patch），支持折线图、散点图、条形图、直方图、箱线图、饼图、热力图、3D 图等。Seaborn 则在其上封装了更贴统计学语境的接口与主题，用于快速完成带分组的统计图形。本章将带你从“画出第一条线”走到“用 Seaborn 做一组带分面/分组的统计报告图”，为后续 Pandas、Statsmodels、sklearn 各章的图表输出打下基础。

### 学习目标

学完本章，你应该能够：

- 1. 使用 matplotlib.pyplot 绘制折线图、散点图、条形图、饼图、直方图、箱线图、热力图与误差棒图；
- 2. 理解 Figure/Axes 对象模型，会用 plt.subplots 创建并管理多子图，调整 figsize/dpi/tight\_layout/subplots
- 3. 设置全局与局部字体、标题、轴标签、刻度、范围、网格、图例，以及颜色/线型/线宽/标记等样式参数；
- 4. 用 plt.savefig 输出高分辨率图片，并知道常见图片格式与 DPI 语义；
- 5. 使用 Seaborn 的内置主题、histplot、boxplot、violinplot、heatmap、FacetGrid 与 pairplot 快速做统计可视化；
- 6. 完成一个“绘图最佳实践”综合案例，把多子图、样式、平滑与分组分析、相关矩阵热力图串起来。

### 案例索引

| 案例卡                                 | 所在节                           | 技术点                              | 难度  |
|-------------------------------------|-------------------------------|----------------------------------|-----|
| 1. 用 subplots 画“成绩四连图”——折线/柱状/直方/箱线 | <a href="#">01 基本绘图</a>       | subplots、四种基础图                   | ★★☆ |
| 2. Seaborn 统计图与分面——用箱线图 + 提琴图比较班级成绩 | <a href="#">03 Seaborn 美化</a> | boxplot / violinplot / FacetGrid | ★★☆ |
| 3. 中文字体与保存——做一张可以交作业的报告图            | <a href="#">02 图窗布局与排版</a>    | 中文字体、tight_layout、savefig        | ★★☆ |
| 4. 传感器信号分析与分组报告图（综合案例）              | <a href="#">04 综合案例</a>       | 多子图 + Seaborn + 滑动平均             | ★★★ |

### 先修要求与运行环境

- 熟悉 Python 基础语法；熟悉 NumPy 数组；了解 Pandas 的 DataFrame 会更轻松（第 4 章内容）。
- 安装 Python 3.10+ 与可视化相关库：

```
pip install matplotlib seaborn pandas numpy
# 若需要做更多统计图（本教材第 7 章会用到），可再加装
pip install statsmodels
```

- 示例数据集 (tips / flights) 由 Seaborn 内置，首次使用会自动下载；请确保能访问 GitHub 或在本地已缓存。

本章目录

| 小节            | 文件                               | 内容                                                    |
|---------------|----------------------------------|-------------------------------------------------------|
| 01 基本绘图       | <a href="#">01-基本绘图.md</a>       | 折线/散点/条形/饼图/直方图/箱线图/热力图/误差棒/极坐标/3D/等高线                |
| 02 图窗布局与排版    | <a href="#">02-图窗布局与排版.md</a>    | Figure/Axes、subplots、figsize/dpi、字体、坐标轴、网格、图例、savefig |
| 03 Seaborn 美化 | <a href="#">03-Seaborn 美化.md</a> | 主题、统计图、FacetGrid/PairGrid、heatmap 等                   |
| 04 综合案例       | <a href="#">04-综合案例.md</a>       | 传感器信号分析报告图（多子图 + Seaborn）                             |
| 05 常见误区与技巧    | <a href="#">05-常见误区与技巧.md</a>    | 高频易错点、性能/调试/自测清单                                      |

数学预备与附录

本章数学很轻，主要用到：

- **F 通用速查**：F.1 记号、F.5 可视化原则（一图一信息、色盲友好）；
- 统计图的数学背景见 **C.1 描述统计**（直方图/箱线图/相关系数图）；
- [打开附录 F](#) | [附录索引](#)

画图前看 F.5 的 4 条原则，避免“图多没信息”。

练习与上机入口

- [本章练习 \(exercises/\)](#)：作业题、自测 quiz（14 题、自动评分）、参考答案。
- [本章上机 \(lab/\)](#)：循序渐进的上机 notebook，含综合任务与检查清单。
- [本章参考与延伸阅读 \(references.md\)](#)：官方文档、精品教程、习题集、中文资料。
- [教学说明 \(teaching.md\)](#)：课时安排、重点难点、考核建议（教师用）。

建议课时

| 环节 | 学时     | 对应内容                         |
|----|--------|------------------------------|
| 讲课 | 3-4 学时 | 01-03 正文 + 04 案例讲解           |
| 上机 | 2-3 学时 | lab/ 逐题完成；课后完成 exercises/ 作业 |

使用说明

- **学生**：先读 01-03 正文并运行代码 → 完成 lab 上机 → 提交 exercises 作业 → 自测 quiz 检验。遇到中文字体问题时，用正文第 2 节的 rcParams 设置。
- **教师**：按 teaching.md 的课时表讲；lab 可作为上机课内容；exercises 中的 quiz 带自动评分，可直接回收。

## 01 基本绘图：折线、散点、条形、饼图、直方图、箱线图 etc

本节对应原版 5.1 的内容，并增补学习目标、常见误区、思考题与延伸阅读。配套图：images/basic\_plots.png (由 build/make\_chap5\_figures.py 生成)。

### 本节目标

- 会用 matplotlib.pyplot 绘制 6 种最常用的基础图表；
- 理解面向对象接口 (fig / ax) 与 “函数式” 接口 (plt.plot) 的关系；
- 会为图表添加标题、坐标轴标签、图例；
- 接触热力图、误差棒图、极坐标、3D、等高线等扩展图型。

### 先修

- Python 基础语法；NumPy 数组 (np.linspace、np.arange、随机数)。

### 官方文档/参考入口

- Matplotlib 官方快速开始：[链接](#)
- Matplotlib 图型画廊：[链接](#)

---

### 1.1 快速开始：import matplotlib.pyplot as plt

Matplotlib 有两种风格：**函数式** (plt.plot, 自动管理当前图) 与 **面向对象** (fig, ax = plt.subplots(), 手动管理 Axes)。本教材优先介绍函数式，第 2 节再切换到面向对象。

```
import numpy as np
import matplotlib.pyplot as plt
```

```
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)
```

```
x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]
```

```
plt.plot(x, y)
plt.title('Simple Plot')
plt.xlabel('x axis')
plt.ylabel('y axis')
plt.show()
```

运行后得到一条折线。若在 Jupyter 中，plt.show() 会把图形显示在输出区；在脚本中则弹出窗口（无窗口环境请用 plt.savefig）。

## 1.2 常见基础图表

### 1.2.1 折线图 (plot)

```
import numpy as np
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]
plt.plot(x, y)
plt.title('Simple Plot')
plt.xlabel('x axis')
plt.ylabel('y axis')
print('x:', x)
print('y:', y)

x: [1, 2, 3, 4, 5]
y: [1, 4, 9, 16, 25]
```

### 1.2.2 散点图 (scatter)

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
x = rng.normal(0, 1, 60)
y = 0.6 * x + rng.normal(0, 0.4, 60)
plt.scatter(x, y, s=28, c='#e07b39', alpha=0.8)
plt.title('Scatter Plot')
print('x min/max:', round(x.min(), 3), round(x.max(), 3))
print('y mean:', round(y.mean(), 3))

x min/max: -2.325 1.96
y mean: 0.08
```

s 控制点面积、c 控制颜色、alpha 控制透明度; np.random.default\_rng(0) 保证结果可复现。

### 1.2.3 条形图 (bar)

```
import matplotlib.pyplot as plt

cats = ['A', 'B', 'C', 'D']
vals = [23, 45, 56, 78]
plt.bar(cats, vals)
plt.title('Bar Chart')
plt.xlabel('Categories')
plt.ylabel('Values')
print('vals:', vals)
print('total:', sum(vals))

vals: [23, 45, 56, 78]
total: 202
```

### 1.2.4 饼图 (pie)

```
import matplotlib.pyplot as plt

labels = 'Frogs', 'Hogs', 'Dogs', 'Logs'
sizes = [15, 30, 45, 10]
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)
plt.title('Pie Chart')
plt.axis('equal')
print('sizes:', sizes, 'sum:', sum(sizes))

sizes: [15, 30, 45, 10] sum: 100
```

autopct 控制百分比格式, startangle 控制起始角度, axis('equal') 保证饼图是正圆。

### 1.2.5 直方图 (hist)

```
import numpy as np
import matplotlib.pyplot as plt

mu, sigma = 0, 0.1
data = np.random.default_rng(5).normal(mu, sigma, 1000)
counts, bins, patches = plt.hist(data, bins=30, density=True)
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')
print('bin count:', len(patches))
print('first/last bin:', round(bins[0], 3), round(bins[-1], 3))

bin count: 30
first/last bin: -0.328 0.311
```

density=True 时纵轴是概率密度 (面积归一为 1); bins 为分箱数。

### 1.2.6 箱线图 (boxplot)

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(10)
data = np.random.normal(0, 1, 200)
data = np.concatenate([data, [10]]) # 加入一个异常值
plt.boxplot(data)
plt.title('Boxplot')
print('outliers > 3:', int((data > 3).sum()))
print('max:', round(data.max(), 2))

outliers > 3: 1
max: 10.0
```

箱线图用五数概括 (Q1、中位数、Q3、上下须) 展示分布, 异常值以离群点显示。

### 1.2.7 热力图 (imshow)

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.default_rng(7).random((10, 10))
plt.imshow(data, cmap='hot', interpolation='nearest')
plt.colorbar()
plt.title('Heatmap Example')
print('matrix shape:', data.shape)

matrix shape: (10, 10)

imshow 适合展示 2D 数值矩阵; cmap 控制配色, colorbar 加色标。
```

### 1.2.8 误差棒图 (errorbar)

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0.1, 4, 0.5)
y = np.exp(-x)
yerr = 0.1 + 0.2 * np.sqrt(x)
xerr = 0.1 * np.ones_like(x)
plt.errorbar(x, y, xerr=xerr, yerr=yerr, fmt='-o')
plt.title('Errorbar Chart Example')
print('n points:', x.size)

n points: 8
```

### 1.2.9 扩展图型: 极坐标、3D、等高线

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 极坐标
r = np.arange(0, 2, 0.01)
theta = 2 * np.pi * r
plt.polar(theta, r)
plt.title('Polar Chart')
print('polar points:', r.size)
plt.close() # 关闭极坐标图, 避免与下面的 3D/等高线共用 Figure

# 3D 曲面
x = np.linspace(-5, 5, 100); y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))
fig3d = plt.figure()
ax = fig3d.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')
```

```
print('surface shape:', Z.shape)
plt.close(fig3d)      # 关闭 3D 图，避免与下面的等高线共用 Figure
```

```
# 等高线（显式用二维 Axes，避免落在 3D 坐标轴上报错）
fig2, ax2 = plt.subplots()
cs = ax2.contourf(X, Y, Z, cmap='viridis')
ax2.contour(X, Y, Z, colors='k')
fig2.colorbar(cs)
print('X,Y,Z shapes:', X.shape, Y.shape, Z.shape)

polar points: 200
surface shape: (100, 100)
X,Y,Z shapes: (100, 100) (100, 100) (100, 100)
```

3D 图需要 `from mpl_toolkits.mplot3d import Axes3D; add_subplot(111, projection='3d')` 表示 1×1 网格中的第一个子图（详见第 2 节）。

Matplotlib 基础图表一览

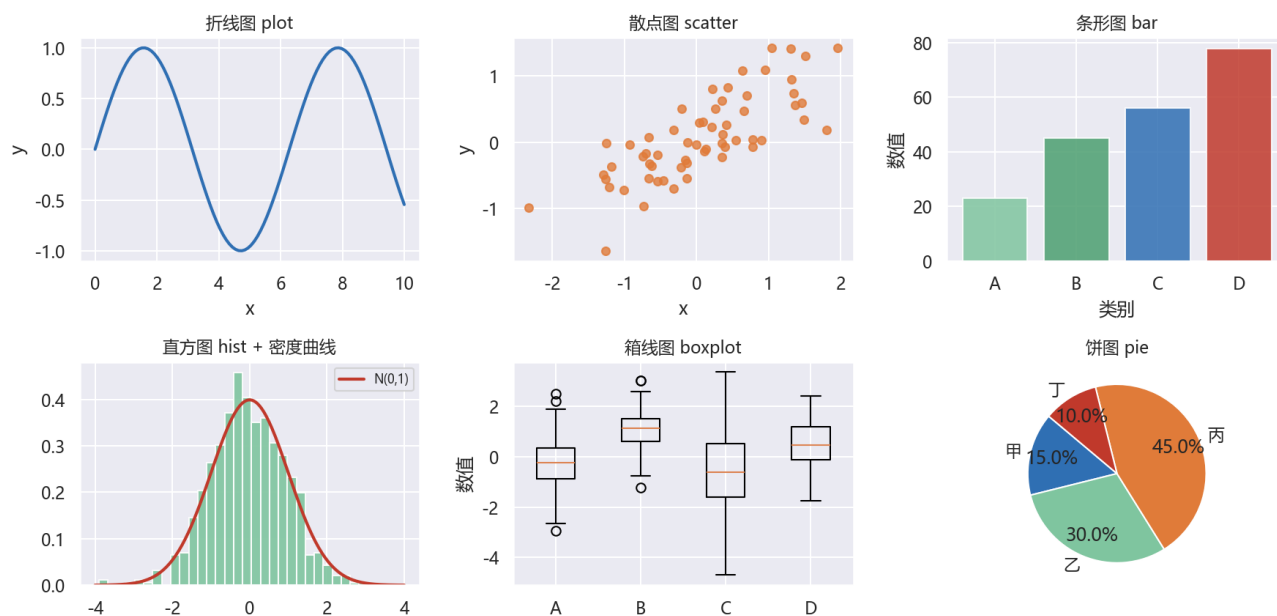


Figure 1: 基础图表一览

## 案例卡 1：用 subplots 画“成绩四连图”——折线/柱状/直方/箱线

### 目标

把一张 4 名学生 × 3 门课的成绩表，用一张 2×2 的 subplots 同时画出四种基础图：折线图、柱状图、直方图、箱线图。学会“先想清楚每格放什么，再往对应的 ax 上画”。

### 代码

```
import numpy as np
import matplotlib.pyplot as plt
```

```
import os

os.makedirs('images', exist_ok=True)
plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

students = ['小明', '小红', '小刚', '小丽']
subjects = ['语文', '数学', '英语']
scores = np.array([
    [88, 92, 79],
    [67, 73, 60],
    [95, 88, 90],
    [72, 80, 83],
])
print('shape:', scores.shape)

fig, axes = plt.subplots(2, 2, figsize=(10, 7))
for i, name in enumerate(students):
    axes[0, 0].plot(subjects, scores[i], marker='o', label=name)
axes[0, 0].set_title('每位学生三科成绩折线')
axes[0, 0].legend(fontsize=8); axes[0, 0].grid(alpha=0.3)

means = scores.mean(axis=0)
axes[0, 1].bar(subjects, means, color=['#4C9F70', '#2F6FB3', '#C0392B'])
axes[0, 1].set_title('各科平均分')
for j, m in enumerate(means):
    axes[0, 1].text(j, m + 1, f'{m:.1f}', ha='center', fontsize=9)

axes[1, 0].hist(scores.ravel(), bins=8, alpha=0.7, color='#7fc59f', edgecolor='white')
axes[1, 0].set_title('全班成绩直方图')
axes[1, 0].set_xlabel('分数'); axes[1, 0].set_ylabel('人数')

axes[1, 1].boxplot([scores[:, j] for j in range(3)], tick_labels=subjects)
axes[1, 1].set_title('各科成绩箱线图')
axes[1, 1].set_ylabel('分数')

fig.suptitle('成绩四连图: 折线/柱状/直方/箱线', fontsize=14)
fig.tight_layout(rect=[0, 0, 1, 0.95])
fig.savefig('images/case1_scores_subplots.png', dpi=150)
print('mean by subject:', np.round(means, 2))
print('max total index:', int(np.argmax(scores.sum(axis=1))))
print('done')
```

### 运行结果 (已运行核验)

```
shape: (4, 3)
mean by subject: [80.5  83.25 78.  ]
```

```
max total index: 2
done
```

讲解

**一句话解释:** `fig, axes = plt.subplots(2, 2)` 相当于买了一张 2×2 的“画板”并把它分成 4 个小画板, 用 `axes[0,0]`、`axes[1,1]` 等分别往里面画, 最后 `savefig` 一次性保存。

- `scores` 是 (4, 3) 的矩阵: 4 行 = 4 名学生, 3 列 = 语文/数学/英语。
- `axes[0, 0].plot(subjects, scores[i], ...)` 中 `scores[i]` 是第 *i* 个学生的三门成绩, 所以水平方向是科目、垂直方向是分数。
- `scores.mean(axis=0)` 沿“学生”这一维折叠, 得到三门课各自平均分; `axis=0` 口诀: 消除第 0 维, 保留第 1 维。
- `scores.ravel()` 把 (4, 3) 展平成 12 个分数, `hist` 画出这 12 个数的分布。
- `boxplot([scores[:, 0], scores[:, 1], scores[:, 2]])` 把三门课各看成一串数据, 画出三个“箱子”, 一眼看出哪门课最稳、哪门课波动大。
- 每个子图都有自己的 `set_title/set_xlabel/set_ylabel`; 全部画完后 `fig.tight_layout()` 让子图不挤在一起, `fig.savefig(...)` 输出到 `images/`。

主要用法 / API

| 需求        | 写法                                                 |
|-----------|----------------------------------------------------|
| 创建 2×2 子图 | <code>fig, axes = plt.subplots(2, 2)</code>        |
| 访问子图      | <code>axes[0, 0] / axes[1, 1]</code>               |
| 折线图       | <code>axes[i, j].plot(x, y, marker='o')</code>     |
| 柱状图       | <code>axes[i, j].bar(x, y)</code>                  |
| 直方图       | <code>axes[i, j].hist(data, bins=n)</code>         |
| 箱线图       | <code>axes[i, j].boxplot([col1, col2, ...])</code> |
| 保存        | <code>fig.savefig('images/x.png', dpi=150)</code>  |

常见错误

- 忘记 `axes` 是 2×2 的 ndarray, 直接 `axes.plot(...)` 会报 `AttributeError`;
- 把不同图都画到同一个 `axes` (比如一直用 `axes[0, 0]`), 导致四个格子只有一个有内容;
- 不调用 `fig.tight_layout()`, 子图标题/标签互相重叠;
- 用 `plt.savefig(...)` 但没先创建“当前图”; 这里建议始终用 `fig.savefig(...)`。

拓展 / 跟练

1. 把子图改为 1×4 或 2×2 之后调换顺序, 观察排版差异;
2. 给柱状图每根柱子加上数值标签 (参考 `text` 用法);
3. 把箱线图改成“每个学生”而不是“每门课”;
4. 把这张图保存为 300 DPI 的 PNG, 并比较文件大小。

常见误区

1. 只调用 `plt.plot` 不调用 `plt.show()` 或 `savefig`: 在脚本中可能不显示也不保存, 图形“消失”。

成绩四连图：折线/柱状/直方/箱线

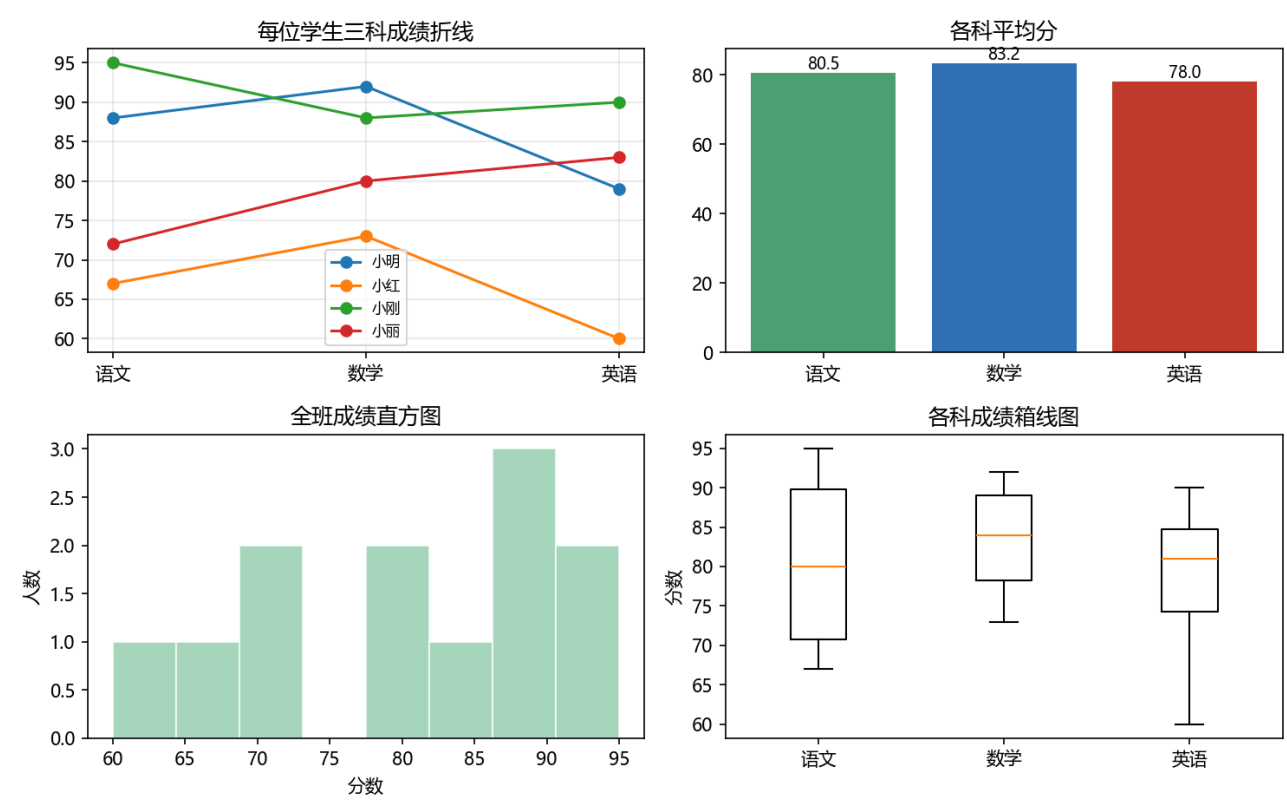


Figure 2: 成绩四连图

2. 对 `imshow` 不加 `colorbar`: 看不出数值颜色映射。
3. `plt.hist` 直接打印可以拿到三个返回值: `counts`, `bins`, `patches`, 很多同学忽略。
4. 误以为 `pie` 的 `axis('equal')` 可省: 不设会变成椭圆。
5. 随机数不设种子: 每次输出不同, 课堂演示/作业请用 `default_rng(seed)`。

## 思考题

1. 为什么 `plt.plot(x, y)` 与 `fig, ax = plt.subplots(); ax.plot(x, y)` 等价? 它们分别管理什么状态?
2. `plt.scatter` 与 `plt.plot(..., marker='o')` 对一维数据有何适用场景差异?
3. 直方图 `density=True` 后, 纵轴还能不能叫 “Frequency”? 为什么?
4. 3D 曲面图绘制时为什么要先 `np.meshgrid`?

## 动手练习（详见 lab）

- 绘制  $y = x^3 - 2x^2 + x$  在  $[-2, 2]$  的折线图, 加网格与图例;
- 用 `default_rng(0)` 生成 100 个标准正态数, 画 30 箱直方图叠加理论密度曲线;
- 用 `tips` 数据集画 `day` 分组箱线图;
- 画一张  $10 \times 10$  随机矩阵热力图, 并对比两种 `cmap`。

## 延伸阅读

- Matplotlib 官方入门: [链接](#)
- Matplotlib 图型画廊 (Gallery): [链接](#)
- Seaborn 官方统计图: [链接](#)

## 02 图窗、布局与排版：Figure/Axes、子图、字体、坐标轴与保存

本节对应原版 5.2 的内容，并增补学习目标、常见误区、思考题与延伸阅读。配套图：images/subplots\_layout.png (由 build/make\_chap5\_figures.py 生成)。

### 本节目标

- 理解 Figure (画布) 与 Axes (坐标区) 的对象层级；
- 会用 plt.subplots 创建多子图并用 ax 逐区绘制；
- 会设置 figsize/dpi, 用 rcParams 做全局字体与字号；
- 会设置轴标签、刻度、范围、网格、图例；
- 会使用 tight\_layout / subplots\_adjust 排版，并掌握 savefig 输出。

### 先修

- 第 1 节的基础图表；NumPy 与 Pandas 基本操作。

### 官方文档/参考入口

- Matplotlib 官方 “面向对象接口”：[链接](#)
- Matplotlib 官方 “文本/字体”：[链接](#)
- Matplotlib 官方 “布局指南”：[链接](#)

---

## 2.1 Figure 与 Axes：两张“画布”

Matplotlib 的核心对象是 **Figure** (整张图) 与 **Axes** (图中一个坐标系)。一个 Figure 可以有多个 Axes。

```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

fig = plt.figure(figsize=(10, 4), dpi=100)
ax = fig.add_subplot(111)          # 111 表示 1x1 网格中的第 1 个
print('figsize:', fig.get_figwidth(), 'x', fig.get_figheight())
print('dpi:', fig.get_dpi())
print('axes count:', len(fig.axes))

figsize: 10.0 x 4.0
dpi: 100
axes count: 1
```

plt.plot(...) 等价于 “在当前 Figure 上自动取一个 Axes 再画”，有复用代码简单、但不易同时画多子的缺点。

## 2.2 子图: plt.subplots

```
import matplotlib.pyplot as plt
```

```
fig, axs = plt.subplots(2, 2)
print('axs shape:', axs.shape)
print('n axes:', axs.size)
for ax in axs.flat:
    ax.plot([1, 2], [2, 1])
print('ok')
```

```
axs shape: (2, 2)
```

```
n axes: 4
```

```
ok
```

axs 是 ndarray; 用 axs[0, 1] 访问指定子区, axs.flat 可顺序遍历。

## 2.3 图窗大小与分辨率: figsize 与 dpi

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8, 6), dpi=100)
plt.plot([1, 2, 3], [4, 3, 2])
# figsize 单位英寸; dpi 为每英寸像素点数
print('figure created')
```

输出: 创建一张 8×6 英寸、100 DPI 的图窗 (像素 800×600)。

## 2.4 全局字体与字号: rcParams

中文标题很常用; 若不做设置, 可能显示为方块。用 rcParams 全局设置即可:

```
import matplotlib as mpl
import matplotlib.pyplot as plt

mpl.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
mpl.rcParams['axes.unicode_minus'] = False # 让负号正常显示
mpl.rcParams['font.size'] = 12
plt.plot([1, 2, 3], [4, 3, 2])
plt.title('标题')
print('after:', mpl.rcParams['font.size'])
```

```
after: 12.0
```

Windows 上 Microsoft YaHei / SimHei 常见; Linux/macOS 可换成 Noto Sans CJK SC 或 Arial Unicode MS。

## 2.5 坐标轴: 标签、范围、刻度

```
import matplotlib.pyplot as plt
```

```
fig, ax = plt.subplots()
```

```

ax.plot([1, 2, 3], [4, 3, 2])
ax.set_xlabel('X Axis Label', fontsize=14)
ax.set_ylabel('Y Axis Label', fontsize=14)
ax.set_xlim(0, 4)
ax.set_ylim(2, 5)
ax.set_xticks([1, 2, 3])
ax.set_yticks([3, 4])
print('xlim:', tuple(float(v) for v in ax.get_xlim()))
print('ylim:', tuple(float(v) for v in ax.get_ylim()))
print('xticks:', [int(v) for v in ax.get_xticks()])

xlim: (0.0, 4.0)
ylim: (2.0, 5.0)
xticks: [1, 2, 3]

```

## 2.6 网格与图例

```

import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([1, 2, 3], [4, 3, 2], label='line 1')
ax.plot([1, 2, 3], [2, 3, 4], label='line 2')
ax.grid(color='gray', linestyle='--', linewidth=0.5)
ax.legend()
print('handles:', len(ax.get_legend().get_texts()))

handles: 2

```

图例 loc 可选 'best'、'upper right'、'lower left' 等; grid 可设颜色/线型/线宽。

## 2.7 排版: tight\_layout 与 subplots\_adjust

tight\_layout 自动让子图不重叠; subplots\_adjust 手动控制边界与间距。

```

import matplotlib.pyplot as plt

fig, axs = plt.subplots(2, 2, figsize=(9, 6))
for ax in axs.flat:
    ax.plot([1, 2, 3], [2, 1, 3])
fig.suptitle('示例', fontsize=14)
fig.tight_layout()  # 自动排版, 推荐
# fig.subplots_adjust(left=0.1, right=0.9, top=0.9, bottom=0.1, wspace=0.3, hspace=0.3)
print('layout ok')

```

输出: layout ok, 运行后得到整洁的 2×2 子图 (见下方示例图)。

## 2.8 保存图像: savefig

```

import matplotlib.pyplot as plt
import os

```

图窗布局与排版: 2×2 子图 + 样式

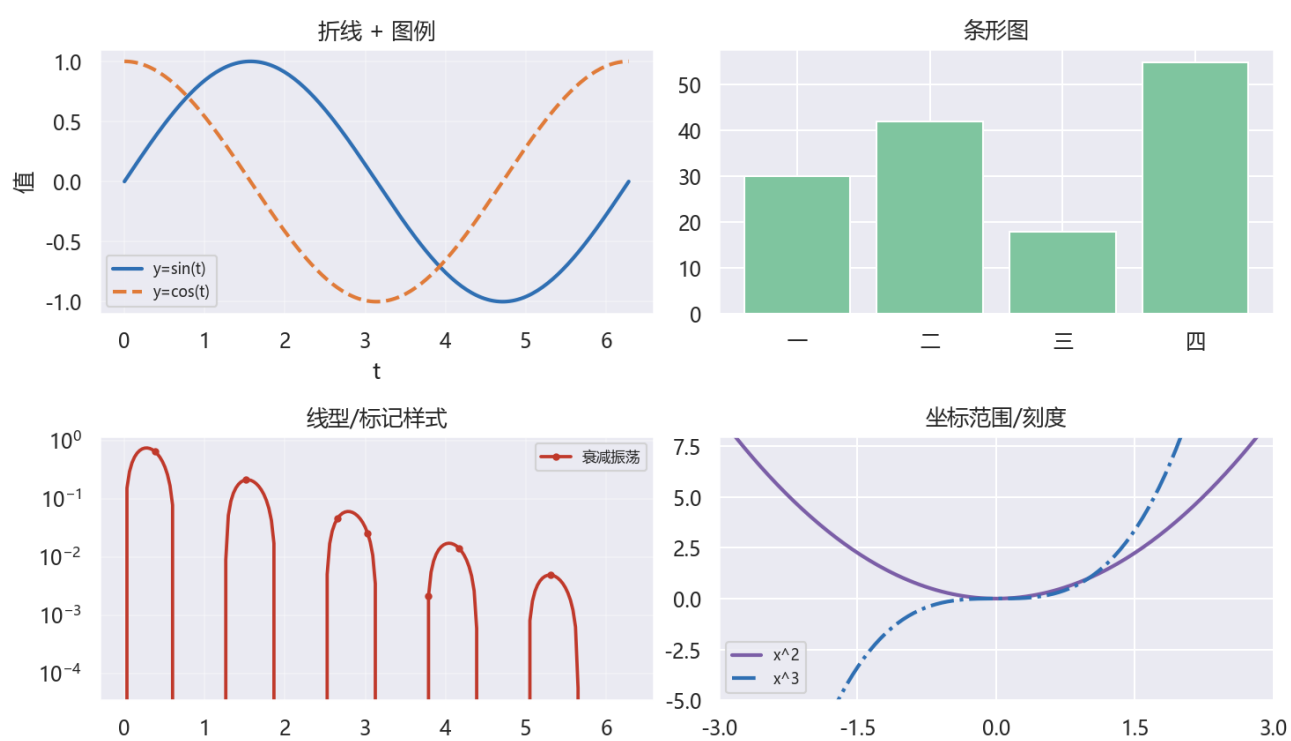


Figure 1: 图窗布局示例

```
plt.figure(figsize=(8, 6), dpi=100)
plt.plot([1, 2, 3], [4, 3, 2])
plt.savefig('my_plot.png', dpi=150)
print('exists:', os.path.exists('my_plot.png'))
```

exists: True

savefig 的 dpi 可单独覆盖图窗 DPI；格式由扩展名决定（png / jpg / pdf / svg 等）。

### 案例卡 3：中文字体与保存——做一张可以交作业的报告图

#### 目标

配置中文字体与负号显示，做一张包含“柱状图 + 折线图”的报告图，并同时保存为高分辨率 PNG 与 PDF。学会“交作业前最后两步：tight\_layout() + savefig”。

#### 代码

```
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)
os.makedirs('images', exist_ok=True)

months = ['1 月', '2 月', '3 月', '4 月', '5 月', '6 月']
sales = np.array([120, 95, 150, 170, 140, 210])
print('months:', len(months), ' max index:', int(np.argmax(sales)))

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].bar(months, sales, color='#2f6fb3')
axes[0].set_title('月度销量柱状图')
axes[0].set_xlabel('月份'); axes[0].set_ylabel('销量')
for i, v in enumerate(sales):
    axes[0].text(i, v + 4, f'{v}', ha='center', fontsize=9)

axes[1].plot(months, sales, marker='o', color='#e07b39', lw=2)
axes[1].set_title('月度销量趋势')
axes[1].set_xlabel('月份'); axes[1].set_ylabel('销量')
axes[1].grid(alpha=0.3)

fig.suptitle('中文字体与保存：作业报告图', fontsize=15)
fig.tight_layout(rect=[0, 0, 1, 0.94])
fig.savefig('images/case3_report.png', dpi=150)
fig.savefig('images/case3_report.pdf')
print('savefig dpi:', 150)
print('done')
```

运行结果（已运行核验）

```
months: 6  max index: 5
savefig dpi: 150
done
```

讲解

**一句话解释：**先告诉 Matplotlib “用哪套中文字体、负号怎么显示”，再画图，最后 `tight_layout` 整理排版、`savefig` 存成文件——这就是“可以交作业”的报告图。

- 前两行 `plt.rcParams['font.sans-serif'] = [...]` 让中文标题、坐标轴标签不再显示成方块；`axes.unicode_minus = False` 避免负号变成“横杠”。
- `sales` 是 6 个月的销量；`np.argmax(sales)` 返回最大值的下标（从 0 数），这里 210 在下标 5。
- `axes[0].bar(...)` 画柱状图；`axes[0].text(i, v + 4, f'{v}', ...)` 在每个柱子顶上标数字，`v + 4` 是为了把文字稍微抬高、不与柱子重叠。
- `axes[1].plot(...)` 画折线图，用 `marker='o'` 标出每个数据点。
- `fig.tight_layout(rect=[0, 0, 1, 0.94])` 让子图和总标题不重叠；`rect` 给最上方的 `suptitle` 留出空间。
- `fig.savefig('images/case3_report.png', dpi=150)` 输出 PNG；再次 `savefig('images/case3_report')` 另存一份 PDF，方便打印或插入报告。

主要用法 / API

| 需求         | 写法                                                                              |
|------------|---------------------------------------------------------------------------------|
| 设中文字体      | <code>plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', ...]</code>         |
| 修复负号       | <code>plt.rcParams['axes.unicode_minus'] = False</code>                         |
| 创建子图       | <code>fig, axes = plt.subplots(1, 2, figsize=(11, 4))</code>                    |
| 子图标题       | <code>axes[i].set_title('...')</code>                                           |
| 整理排版       | <code>fig.tight_layout(rect=[0, 0, 1, 0.94])</code>                             |
| 保存 PNG/PDF | <code>fig.savefig('images/x.png', dpi=150) / fig.savefig('images/x.pdf')</code> |

常见错误

- 只在 `set_theme` 之前设置字体，之后被 `seaborn` 重置（见 03 节）；
- 用 `plt.savefig(...)` 而不是 `fig.savefig(...)`，多子图时可能存错“当前图”；
- 不调用 `tight_layout`，保存出来后标题被裁剪；
- 忘记 `os.makedirs('images', exist_ok=True)`，直接 `savefig('images/...')` 会因目录不存在而报错。

拓展 / 跟练

1. 把主题换成 `plt.style.use('seaborn-v0_8-whitegrid')`，再比较字体是否被覆盖；
2. 给折线加上数值标注与数据点 `annotate`；
3. 分别用 100 / 150 / 300 DPI 保存三张 PNG，比较清晰度与文件大小；
4. 把这张图改成 2×2 报告图（增加直方图与箱线图）。

中文字体与保存：作业报告图

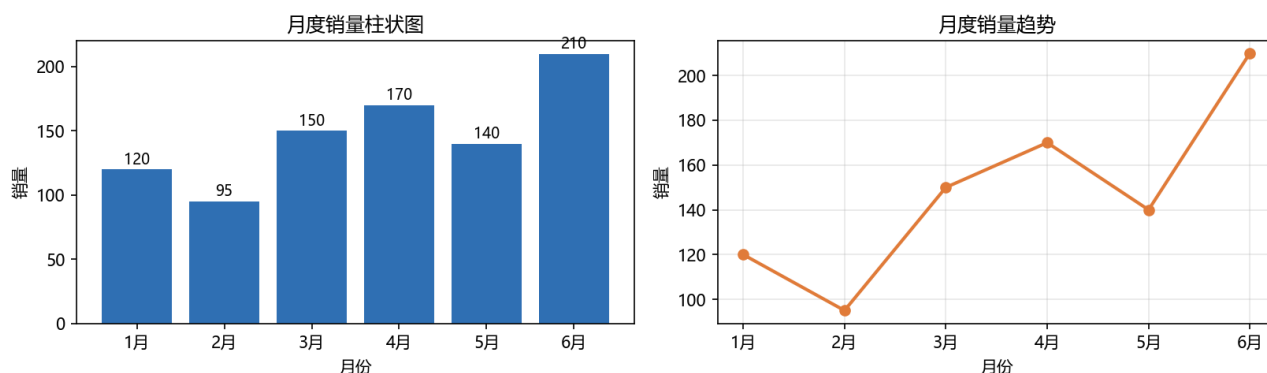


Figure 2: 作业报告图示例

## 常见误区

1. **混用 plt 与 ax 接口**：在面向对象代码中继续用 `plt.title` 可能作用于“当前”轴，子图多了易错。建议一个图中要么纯函数式、要么纯面向对象。
2. **忘记 `tight_layout`**：子图标题、轴标签互相重叠。
3. **`set_xticks` 与 `set_xticklabels` 混用**：前者设刻度位置，后者改刻度文字。
4. **把 `figsize` 单位当像素**：实际单位是英寸，配合 `dpi` 才得到像素。
5. **`rcParams` 在 `seaborn.set_theme` 之后设置**：否则字体被覆盖回 Arial（见第 3 节）。

## 思考题

1. `fig, axs = plt.subplots(3, 4)` 中 `axs.shape` 是多少？如何访问第 2 行第 3 列子图？
2. 为什么 `plt.show()` 不能在无窗口服务器上显示图形？此时用哪种方法输出？
3. `tight_layout` 和 `subplots_adjust` 的区别是什么？各自适合什么场景？
4. 要在每个子图上画不同内容并共用同一 x 轴，`axs.flat` 与显式下标哪种更清晰？

## 动手练习（详见 lab）

- 创建 2×3 子图，分别画 `sin`、`cos`、`tan`、`exp`、`log`、`x^2`；
- 用 `rcParams` 设置全局字号 12，并为每个子图加中文标题；
- 上一题做完后调用 `fig.tight_layout()` 观察差异；
- 用 `savefig` 导出 300 DPI 的 PDF。

## 延伸阅读

- Matplotlib 官方 “Pyplot 教程”：[链接](#)
- Matplotlib 官方 “面向对象 API”：[链接](#)
- Matplotlib 官方 “文本/字体”：[链接](#)

## 03 Seaborn 美化：主题、统计图与分面

本节对应原版 5.3 的内容，并增补学习目标、常见误区、思考题与延伸阅读。配套图：images/seaborn\_style.png (由 build/make\_chap5\_figures.py 生成)。

### 本节目标

- 理解 Seaborn 与 Matplotlib 的关系；
- 会用 `set_theme` / `set_style` 切换全局主题；
- 会用 `histplot`、`kdeplot`、`boxplot`、`violinplot`、`heatmap` 做统计可视化；
- 会用 `FacetGrid` / `pairplot` 做分面与成对图；
- 会与 Matplotlib 的 `subplots` 结合做多子图报告图。

### 先修

- 第 1–2 节的内容；Pandas DataFrame (`load_dataset` 返回的就是 DataFrame)。

### 官方文档/参考入口

- Seaborn 官方教程：[链接](#)
- Seaborn 官方 API：[链接](#)
- Matplotlib 官方图型画廊：[链接](#)

---

## 3.1 Seaborn 是什么

Seaborn 建立在 Matplotlib 之上，提供：更口语化的函数名、可直接接受 DataFrame 列名、内置统计（如 KDE、分组箱线）、以及一套美观默认主题。

**注意：**`sns.set_theme(style="darkgrid")` 会重置 Matplotlib 的 `rcParams`（包括字体），所以**设置主题后要重新设中文字体**，否则中文标题会变方块。

```
import numpy as np
import seaborn as sns
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
np.set_printoptions(precision=4, suppress=True)
sns.set_theme(style="darkgrid")
mpl.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
mpl.rcParams["axes.unicode_minus"] = False
```

```
tips = sns.load_dataset("tips")
print("tips shape:", tips.shape)
print(tips.head(3).to_string(index=False))
```

```
tips shape: (244, 7)
total_bill  tip    sex smoker day   time  size
16.99  1.01  Female      No  Sun  Dinner    2
```

|       |      |      |               |   |
|-------|------|------|---------------|---|
| 10.34 | 1.66 | Male | No Sun Dinner | 3 |
| 21.01 | 3.50 | Male | No Sun Dinner | 3 |

常见主题: darkgrid (默认)、whitegrid、dark、white、ticks。

## 3.2 在子图中使用 Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
sns.lineplot(x="tip", y="total_bill", data=tips, ax=axes[0, 0])
sns.barplot(x="sex", y="total_bill", data=tips, ax=axes[0, 1])
sns.scatterplot(x="total_bill", y="tip", hue="sex", data=tips, ax=axes[1, 0])
sns.histplot(data=tips["total_bill"], ax=axes[1, 1])
plt.tight_layout()
print("subplots ok:", axes.shape)

subplots ok: (2, 2)
```

把 data= 传入 DataFrame, 并给 x/y/hue 传列名, Seaborn 自动分组。

## 3.3 常用统计图

### 3.3.1 频率直方图 + 核密度线

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
sns.histplot(tips, x="total_bill", kde=True)
plt.show()
```

### 3.3.2 概率密度曲线

```
sns.kdeplot(tips["total_bill"], fill=True)
plt.show()
```

新版 seaborn 用 fill=True (旧版 shade=True 已弃用)。

### 3.3.3 箱线图与提琴图

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))
sns.boxplot(x="day", y="total_bill", data=tips, ax=axes[0])
sns.violinplot(x="day", y="total_bill", data=tips, ax=axes[1])
print("boxes:", len(axes[0].containers), "violins:", len(axes[1].collections))
```

```
boxes: 1 violins: 4
```

### 3.3.4 热力图

```
import seaborn as sns
import matplotlib.pyplot as plt

flights = sns.load_dataset("flights")
flights_pivot = flights.pivot_table(index="month", columns="year", values="passengers", obs
sns.heatmap(flights_pivot, annot=True, fmt=".0f")
print("pivot shape:", flights_pivot.shape)

pivot shape: (12, 12)
```

### 3.3.5 成对关系图

```
import seaborn as sns

tips = sns.load_dataset("tips")
g = sns.pairplot(tips, hue="day")
print("pairplot grid shape:", g.axes.shape)

pairplot grid shape: (3, 3)
```

## 3.4 分面与多图: FacetGrid / PairGrid

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
g = sns.FacetGrid(tips, col="day", col_wrap=4)
g.map(sns.lineplot, "total_bill", "tip")
g.add_legend()
print("FacetGrid axes:", len(g.axes.flat))

FacetGrid axes: 4
```

FacetGrid 按列取值把数据切成多个子图；col\_wrap 控制每行放几个。PairGrid 则用于变量两两成对。

## 案例卡 2：Seaborn 统计图与分面——用箱线图 + 提琴图比较班级成绩

### 目标

用一份离线可复现的“班级成绩” DataFrame，画箱线图与提琴图对比三个班的成绩分布，再用 FacetGrid 按班级分面画直方图。学会“先构造数据，再用 seaborn 一句画出统计图，最后分面对比”。

### 代码

```
import numpy as np
import pandas as pd
```

Seaborn 统计图与美化

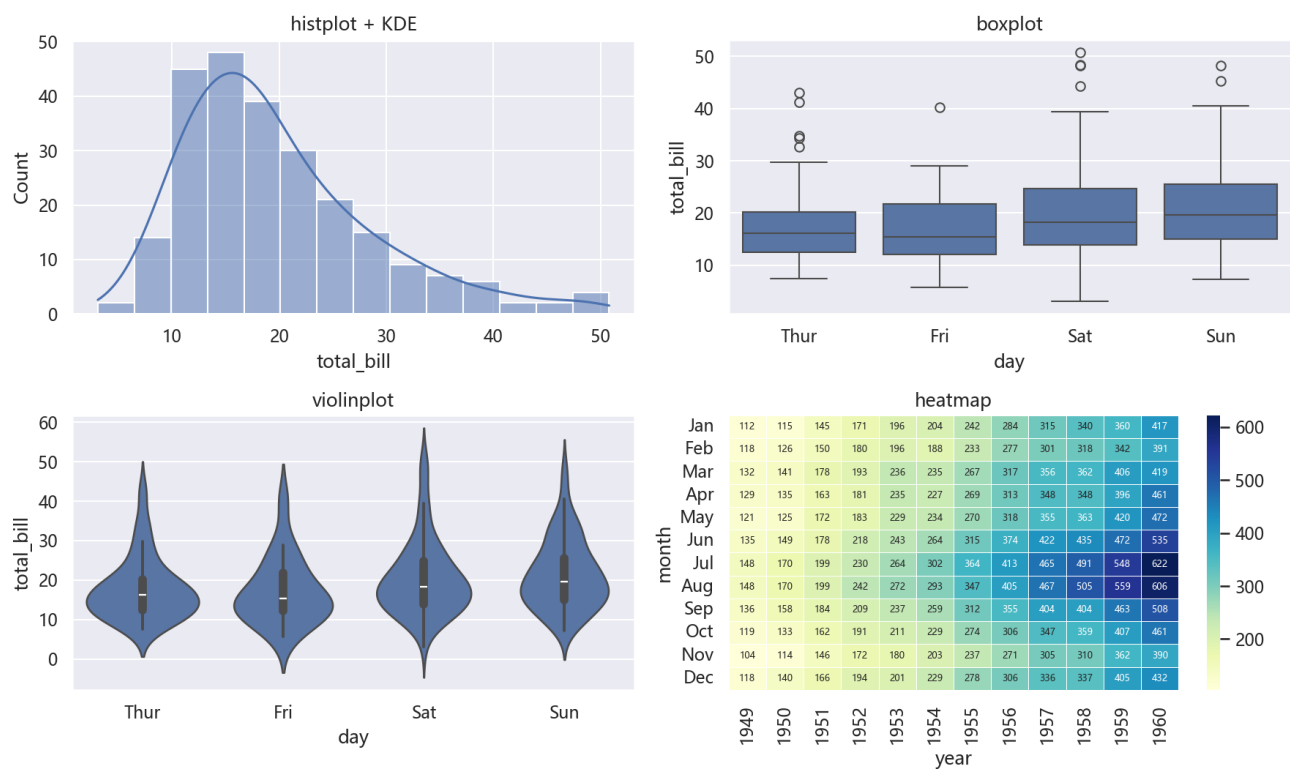


Figure 1: Seaborn 统计图与美化

```
import matplotlib.pyplot as plt
import seaborn as sns
import os
from matplotlib import rcParams

os.makedirs('images', exist_ok=True)
sns.set_theme(style='darkgrid')
rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

rng = np.random.default_rng(7)
n = 60
df = pd.DataFrame({
    '班级': np.repeat(['1 班', '2 班', '3 班'], n),
    '成绩': np.concatenate([
        rng.normal(72, 8, n),
        rng.normal(80, 9, n),
        rng.normal(88, 7, n),
    ]),
})
print('shape:', df.shape)
print('means:', np.round(df.groupby('班级')['成绩'].mean().to_numpy(), 2))

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
sns.boxplot(x='班级', y='成绩', data=df, ax=axes[0])
sns.violinplot(x='班级', y='成绩', data=df, ax=axes[1])
axes[0].set_title('班级成绩箱线图')
axes[1].set_title('班级成绩提琴图')
for ax in axes:
    ax.set_xlabel('班级'); ax.set_ylabel('成绩')
fig.tight_layout()
fig.savefig('images/case2_class_box_violin.png', dpi=150)

g = sns.FacetGrid(df, col='班级', col_wrap=3, height=2.5, aspect=1.2)
g.map(sns.histplot, '成绩', bins=12)
g.set_titles('{col_name}')
g.savefig('images/case2_class_facet.png', dpi=150)
print('facet axes:', len(g.axes.flat))
print('done')
```

### 运行结果 (已运行核验)

```
shape: (180, 2)
means: [70.33 79.25 86.33]
facet axes: 3
done
```

讲解

**一句话解释：**boxplot 和 violinplot 都是“把一组数画成一个形状”来比大小，前者用箱子 + 须看中位数和异常，后者用“胖瘦”看密度；FacetGrid 则是“按班级切开，横着排三个小图”。

- `np.repeat(['1 班','2 班','3 班'], n)` 生成 3 个班、每班 `n=60` 条的“班级”列；三个 `rng.normal(...)` 分别制造不同均值的成绩列，所以三个班天然可分。
- `df.shape` 是 `(180, 2)`：180 行 = `60×3`，2 列 = 班级、成绩。
- `df.groupby('班级')['成绩'].mean()` 按班求平均，`to_numpy()` 取成数组后 `np.round(..., 2)` 保证输出两位小数稳定。
- `sns.boxplot(x='班级', y='成绩', data=df, ax=axes[0])`：把 data 交给 seaborn，它自己按 x 分组，画出 3 个箱子。
- `sns.violinplot(...)` 画提琴图，轮廓越宽表示该分数段的人数越多，能看出“哪个班更集中”。
- `g = sns.FacetGrid(df, col='班级', col_wrap=3)` 按“班级”分成 3 个子图；`g.map(sns.histplot, '成绩', bins=12)` 往每个子图画成绩直方图。

主要用法 / API

| 需求     | 写法                                                                                           |
|--------|----------------------------------------------------------------------------------------------|
| 分组箱线图  | <code>sns.boxplot(x='班级', y='成绩', data=df, ax=axes[0])</code>                                |
| 分组提琴图  | <code>sns.violinplot(x='班级', y='成绩', data=df, ax=axes[1])</code>                             |
| 分面直方图  | <code>g = sns.FacetGrid(df, col='班级', col_wrap=3); g.map(sns.histplot, '成绩', bins=12)</code> |
| 设置分面标题 | <code>g.set_titles('{col_name}')</code>                                                      |
| 保存分面图  | <code>g.savefig('images/x.png', dpi=150)</code>                                              |
| 主题     | <code>sns.set_theme(style='darkgrid')</code> （之后重设字体）                                        |

常见错误

- `set_theme` 之后不重设中文字体，中文标题变方块；
- 把 `data=` 的 DataFrame 与 x/y 列名写错，或忘了 `data=` 导致按数组而非按列分组；
- 用旧版 `shade=True`，新版应写 `fill=True`；
- FacetGrid 用 `col=` 而不是 `hue=` 时，想加的图例可能不出现，需调用 `g.add_legend()`（本例用 `set_titles`）。

拓展 / 跟练

1. 把 boxplot 换成 `sns.boxplot(..., hue='...')`，再加一个“性别”列观察副分组；
2. 用 `sns.kdeplot(data=df, x='成绩', hue='班级')` 画三条密度曲线叠加；
3. 给分面图加 `g.savefig(...)` 并调整 `height/aspect`；
4. 把数据换成真实 tips 数据集，比较 day 分组的箱线/提琴图。

常见误区

1. `set_theme` 之后不重设中文字体：中文标题变方块。

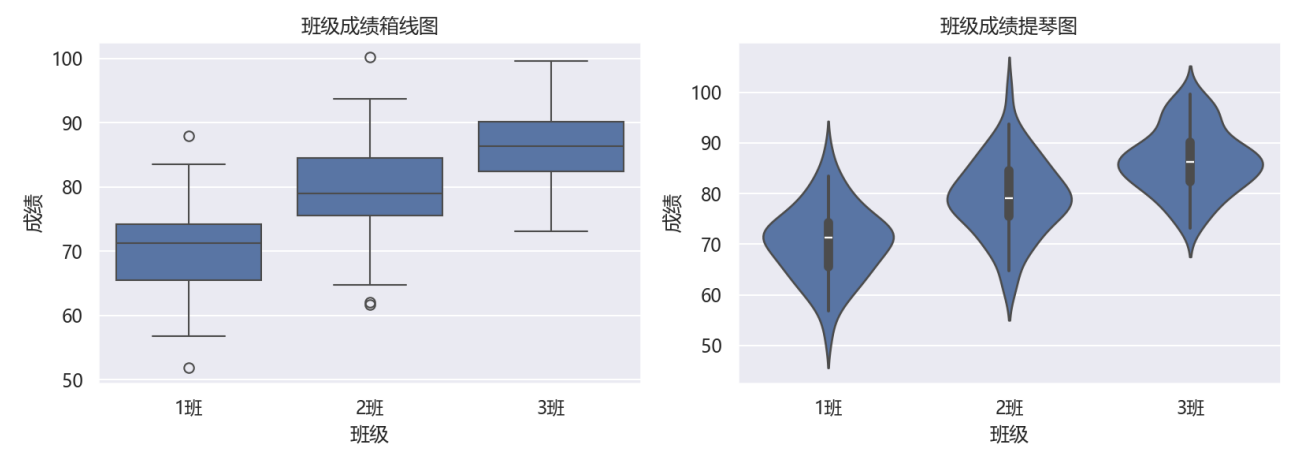


Figure 2: 箱线图与提琴图

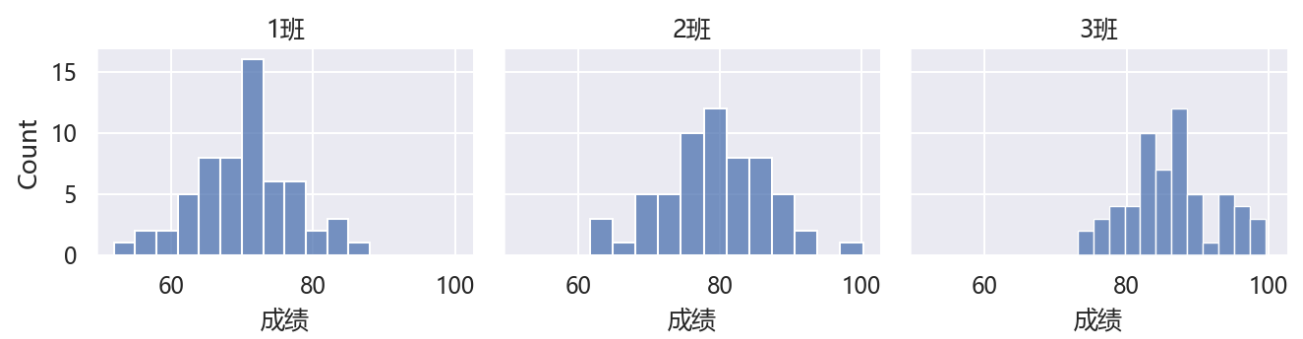


Figure 3: 班级成绩分面图

2. 把旧版 `shade=True` 当新 API: 新版本改为 `fill=True`, 旧参数已弃用。
3. `sns.boxplot` 返回的 `ax` 与 Matplotlib 子图混用 `plt.title`: 多个子图时易错。
4. `load_dataset` 依赖网络: 若要离线教学, 可先用 `pd.DataFrame(...)` 构造样例数据。
5. 给 heatmap 的 `fmt="d"` 传浮点数据: 应改用 `fmt=".0f"` 或 `.2f`。

## 思考题

1. `sns.set_theme` 与 `plt.style.use` 有何异同?
2. `boxplot` 与 `violinplot` 分别适合什么场景?
3. 为什么 `FacetGrid` 比手动 `subplots` 循环更适合 “按类别分面”?
4. 在 Seaborn 图中能否继续用 Matplotlib 的 `set_xticks`? 如何操作?

## 动手练习（详见 lab）

- 用 `tips` 画 `day` 分组的箱线图, 并切换 `whitegrid / ticks` 主题对比;
- 用 `flights` 透视表画热力图, 并自定义 `cmap`;
- 用 `penguins` 数据集做一对 `pairplot`;
- 用 `FacetGrid` 按 `sex` 分面画 `total_bill` 的直方图。

## 延伸阅读

- Seaborn 官方教程: [链接](#)
- Seaborn 官方 API: [链接](#)
- Matplotlib + Seaborn 结合示例: [链接](#)

## 04 综合案例：传感器信号分析与分组报告图

本章新增综合案例，用于把 01–03 的知识串起来：**基础图表 + 多子图/排版 + Seaborn 统计图**。所有代码均可在 `matplotlib ≥ 3.8`、`seaborn ≥ 0.13` 环境运行；配图由 `make_chap5_figures.py` 生成。

### 案例背景

假设你拿到一段“传感器信号”采样（含噪声）与一批分组测量值，需要输出一张**数据分析报告图**：

1. 左上：时间序列折线图，把原始信号与滑动平均后的平滑信号叠在一起；
2. 右上：信号幅值直方图 + 核密度估计（Seaborn）；
3. 左下：对照组 / 实验组的分组散点图；
4. 右下：几个变量之间的相关矩阵热力图。

这正是第 1 章的“数据能画出来” + 第 2 章的“多子图排版” + 第 3 章“Seaborn 统计图”的联合应用。

### 步骤 1：生成合成信号并平滑

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
np.set_printoptions(precision=4, suppress=True)

rng = np.random.default_rng(2025)  # 固定种子，保证可复现
n = 400
t = np.linspace(0, 20, n)
signal = 3 * np.sin(2 * np.pi * 0.25 * t) + 0.6 * np.cos(2 * np.pi * 1.1 * t) + 0.8
y = signal + rng.normal(0, 0.5, n)
smoothed = pd.Series(y).rolling(window=15, center=True).mean().to_numpy()

print(" 信号长度 n =", n)
print(" 原始信号  均值 =", round(float(y.mean()), 3), " 标准差 =", round(float(y.std()), 3))
print(" 平滑信号  均值 =", round(float(np.nanmean(smoothed)), 3))
print(" 平滑信号 前 5 个 =", np.round(smoothed[:5], 3))

信号长度 n = 400
原始信号  均值 = 0.795  标准差 = 2.244
平滑信号  均值 = 0.797
平滑信号 前 5 个 = [nan nan nan nan nan]

rolling(window=15, center=True) 做中心化滑动平均；两端窗口不满时为 NaN，属正常现象。
```

## 步骤 2: 生成分组数据与相关矩阵

```
rng2 = np.random.default_rng(1)
group = rng2.choice([" 对照组", " 实验组"], size=250, p=[0.5, 0.5])
val = np.where(group == " 实验组", rng2.normal(5.2, 1.1, 250), rng2.normal(4.0, 1.0, 250))

df_corr = pd.DataFrame({
    "x1": rng2.normal(0, 1, 300),
    "x2": rng2.normal(0, 1, 300),
    "x3": rng2.normal(0, 1, 300),
})
df_corr["x4"] = 0.8 * df_corr["x1"] + rng2.normal(0, 0.6, 300)
corr = df_corr.corr()

print(" 对照组均值 =", round(float(val[group == " 对照组"].mean()), 3))
print(" 实验组均值 =", round(float(val[group == " 实验组"].mean()), 3))
print(" 相关系数 x1-x4 =", round(float(corr.loc["x1", "x4"]), 3))
print(" 相关系数 x1-x2 =", round(float(corr.loc["x1", "x2"]), 3))
print()
print(corr.round(2).to_string())
```

对照组均值 = 3.874

实验组均值 = 5.274

相关系数 x1-x4 = 0.819

相关系数 x1-x2 = -0.097

|    | x1    | x2    | x3    | x4    |
|----|-------|-------|-------|-------|
| x1 | 1.00  | -0.10 | 0.06  | 0.82  |
| x2 | -0.10 | 1.00  | -0.05 | -0.07 |
| x3 | 0.06  | -0.05 | 1.00  | 0.07  |
| x4 | 0.82  | -0.07 | 0.07  | 1.00  |

因为构造时 x4 由 x1 主导, 所以 x1-x4 相关系数约 0.8, 热力图一眼可辨。

## 步骤 3: 绘制 2×2 报告图

本步依赖步骤 1/2 中定义的 t、y、smoothed、group、val、corr, 请按顺序运行。

```
# 中文字体 (注意: set_theme 会覆盖, 这里放在绘图前重新设置)
import matplotlib as mpl
mpl.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
mpl.rcParams["axes.unicode_minus"] = False

fig, axes = plt.subplots(2, 2, figsize=(12, 8.5))
fig.suptitle(" 综合案例: 传感器信号观测与分组分析", fontsize=15)

axes[0, 0].plot(t, y, color="#c8d8e8", lw=1, label=" 原始信号")
axes[0, 0].plot(t, smoothed, color="#c0392b", lw=2.2, label=" 滑动平均 (窗口 =15)")
axes[0, 0].set_title(" 时间序列: 原始信号 vs 平滑")
axes[0, 0].set_xlabel(" 时间 t"); axes[0, 0].set_ylabel(" 幅值")
```

```
axes[0, 0].legend(fontsize=8); axes[0, 0].grid(alpha=0.3)

sns.histplot(x=y, kde=True, color="#2f6fb3", ax=axes[0, 1])
axes[0, 1].set_title(" 信号幅值分布")
axes[0, 1].set_xlabel(" 幅值")

axes[1, 0].scatter(group[group == " 对照组"], val[group == " 对照组"],
                    color="#7fc59f", s=22, alpha=0.75, label=" 对照组")
axes[1, 0].scatter(group[group == " 实验组"], val[group == " 实验组"],
                    color="#e07b39", s=22, alpha=0.75, label=" 实验组")
axes[1, 0].set_title(" 分组散点图")
axes[1, 0].set_xlabel(" 分组"); axes[1, 0].set_ylabel(" 测量值")
axes[1, 0].legend(fontsize=8); axes[1, 0].grid(alpha=0.3)

sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1,
             ax=axes[1, 1], annot_kws={"fontsize": 9})
axes[1, 1].set_title(" 变量间相关矩阵热力图")

fig.tight_layout(rect=[0, 0, 1, 0.95])
fig.savefig("case_study.png", dpi=150)
print(" 已保存 case_study.png")

输出：已保存 case_study.png，并得到下图。
```

## 步骤 4：再存一份高分辨率 PDF

本步依赖步骤 3 定义的 `fig`，可在画出报告图后直接另存一份 300 DPI 的 PDF，便于打印或插入文档。

```
import os
os.makedirs('images', exist_ok=True)
fig.savefig('images/case_study_report.pdf', dpi=300)
print('pdf saved, dpi 300')

pdf saved, dpi 300
```

小提示：savefig 的 dpi 只影响输出文件的分辨率，不改变 fig 本身的 dpi；想要画布本身更精细，可在 `plt.subplots(figsize=..., dpi=...)` 里设置。

## 分析与延伸

- **左上**：原始信号噪声明显，滑动平均后波形更平滑——直观体会“平滑窗口”对信噪比的影响；
- **右上**：信号幅值近似正态分布（叠加噪声的多频正弦叠加），KDE 与直方图吻合；
- **左下**：实验组测量值整体高于对照组，散点图可直接看出两组的中心差异；
- **右下**：x1 与 x4 相关系数约 0.8，热力图让“共线性”一目了然。

## 拓展任务

1. 把平滑窗口从 15 改为 5 / 31，比较左上图平滑程度与失真；
2. 用 `sns.kdeplot` 分开画对照/实验两组的密度曲线，观察重叠区域；
3. 把相关矩阵改用 `sns.clustermap` 展示变量聚类；

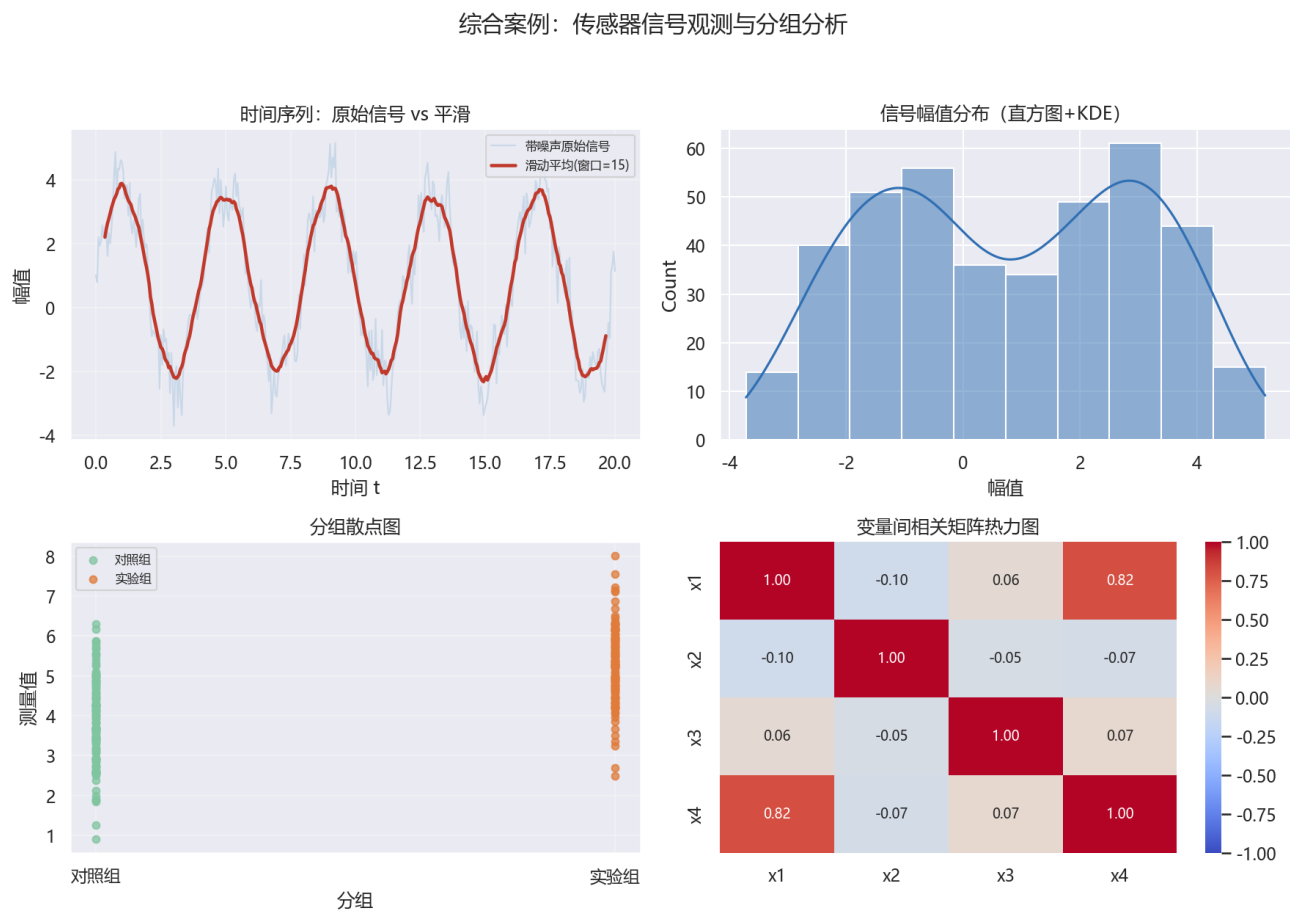


Figure 1: 综合案例报告图

4. 将报告图导出为 PDF，并比较不同 dpi 的文件大小。

## 延伸阅读

- Matplotlib 多子图: [链接](#)
- Seaborn 分布图: [链接](#)
- Pandas rolling: [链接](#)

## 05 常见误区与技巧

本章为新增“查漏补缺”页：把学生最容易踩的坑集中列出，并给出正确的写法和性能技巧。

### 1. 高频误区速查表

| #  | 误区 / 错误写法                              | 正确做法                                                     | 原因                            |
|----|----------------------------------------|----------------------------------------------------------|-------------------------------|
| 1  | 只 plt.plot 不 plt.show()/savefig        | 脚本中用 plt.savefig; 交互环境用 plt.show()                       | 图可能“画了不显示/不保存”                |
| 2  | 面向对象代码里混用 plt.title                    | 用 ax.set_title(...)                                      | 多子图时 plt 作用于当前轴, 易错           |
| 3  | 忘记 ax.set_xticks 与 set_xticklabels 的区别 | 前者设位置, 后者改文字                                             | 两个函数语义不同                      |
| 4  | 误以为 figsize 是像素                        | 单位是英寸, 需乘 dpi 得到像素                                       | 单位语义错误                        |
| 5  | rcParams 在 seaborn set_theme 之前设置      | 主题之后重新设置字体                                               | set_theme 会重置 font.sans-serif |
| 6  | 对 heatmap 传 fmt="d" 而数据是浮点             | 改用 fmt=".0f" 或 .2f                                       | 格式码与数据类型不匹配                   |
| 7  | 用旧版 shade=True                         | 用 fill=True                                              | 新版参数已改名                       |
| 8  | plt.subplots(2,2) 后用 plt.subplot() 再画  | 统一用返回的 axs.flat                                          | 两个接口混用会叠加到错误的轴                |
| 9  | 忘记 np.meshgrid 就画 3D 曲面                | 先 np.meshgrid 生成 2D 网格                                   | 曲面需要网格化的 X, Y, Z              |
| 10 | 误以为一张图只能有一个 Figure                     | 一个脚本可多 Figure; 用 plt.figure() 新建                         | 状态管理理解偏差                      |
| 11 | 中文标题变方块还不设字体                           | 设 rcParams["font.sans-serif"] = ["Microsoft YaHei", ...] | 默认字体不含中文字形                    |
| 12 | 频繁 plt.show() 后又想改 rcParams            | 先设全局参数再绘图                                                | rcParams 影响后续创建的对象            |

### 2. 原理级难点：面向对象 vs 函数式

```
# 函数式：简单，但一张图只有一个“当前轴”
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False
np.set_printoptions(precision=4, suppress=True)

plt.plot([1, 2, 3], [1, 4, 9])
plt.title("one")

# 面向对象：显式管理多个轴
```

```
fig, axs = plt.subplots(2, 2)
axs[0, 0].plot([1, 2], [2, 1])
axs[0, 0].set_title("topleft")
axs[1, 1].scatter([1, 2], [3, 4])
```

经验：单图且临时探索用函数式；多子图/出版图用面向对象。二者混用是学生最常见的返工原因。

### 3. 性能与排版技巧

- 优先 `tight_layout()`：比反复 `subplots_adjust` 手工试更快；
- 用 `ax.set_xticks/set_xticklabels` 控制刻度：避免默认 0,0.5,1 与数据不匹配；
- 大量子图用 `mosaic / GridSpec`：fig, axs = plt.subplots 只支持均匀网格；不均匀用 `GridSpec` 更灵活；
- 图片给 dpi：论文图建议 300 DPI，屏幕演示 100–150 DPI；
- 复用同一 fig 与 ax：一个循环里不断 `plt.figure()` 会开几十个窗口，消耗内存；
- `savefig` 前先 `fig.tight_layout()`，否则标题会被裁剪。

### 4. 调试建议

1. 中文变方块：先检查 `font.sans-serif` 是否在用中文字体，再看 `axes.unicode_minus`；
2. 图形为空/只显示一张：确认是否用了 `plt.figure()` 后又 `plt.subplot` 建了新轴；
3. 子图重叠：调用 `fig.tight_layout()`；
4. 坐标范围不对：打印 `ax.get_xlim() / ax.get_ylim()` 调试；
5. 图例不出现：检查 `plot(label=...)` 是否每一条都有 `label`，再看 `legend()`；
6. 保存的文件打不开：确认有写权限，扩展名正确，且 `savefig` 前没有 `close`。

### 5. 本章小结（自测）

- ☐ 能说出 Figure 与 Axes 的关系；
- ☐ 能创建 2×2 子图并在指定位置画图；
- ☐ 能用 `rcParams` 让中文正常显示；
- ☐ 能设置颜色/线宽/线型/标记/图例/网格；
- ☐ 能用 `savefig` 输出 300 DPI 图片；
- ☐ 能用 Seaborn 画 `histplot / boxplot / heatmap / FacetGrid`；
- ☐ 能独立完成 04 综合案例的报告图。

### 延伸阅读

- Matplotlib 官方 FAQ：[链接](#)
- Matplotlib 官方 “Cheatsheets”：[链接](#)
- Seaborn 官方 “Choosing color palettes”：[链接](#)